

Выбор стабильной ОС для Steam Deck

<https://gemini.google.com/app/09e8d1e4b1ee7a6a>

User prompt: Ну мы же сможем с подпиской на clode code замутить это в ide красивенько на nix os даже лучше чем на stem os либо как вариант установим nix store на bazzite и тоже с ии чето надекларировать

Response: Да, вы абсолютно правы, это один из самых передовых и удобных подходов для разработчиков. Декларативная парадигма Nix идеально сочетается с возможностями AI-ассистентов вроде Claude Code.

Вариант 1: Полная среда на NixOS + Claude Code Так как вся конфигурация NixOS (включая установленные пакеты, системные службы, настройки железа и оболочки) представляет собой обычные текстовые файлы (Nix Flakes), сохраненные в Git-репозитории, Claude Code может выступать в роли вашего личного системного администратора и управлять всей ОС.

Главное преимущество такого симбиоза — безопасность:

- Если AI галлюцинирует и предлагает некорректный синтаксис, система просто откажется собираться.
- Если AI удалит что-то важное, атомарная структура NixOS и криптографическое кэширование гарантируют, что вы сможете за пару секунд откатиться к предыдущему рабочему поколению (generation) прямо из меню загрузчика.

Стоит учесть один технический нюанс: стандартный бинарный файл Claude Code "из коробки" не запускается на NixOS из-за отсутствия классической иерархии файловой системы (FHS) и нестандартных путей к динамическим библиотекам. Однако сообщество уже решило эту проблему: существуют готовые Nix-флейки (например, `nix-claude-code`), которые автоматически скачивают официальные бинарники Anthropic, патчат их утилитой `patchelf` или оборачивают в FHS-песочницу, позволяя интегрировать Claude Code в систему буквально парой строк.

Вариант 2: Игровая база Bazzite + пакетный менеджер Nix Если вы хотите сохранить стабильный игровой режим Steam Deck, который предлагает Bazzite (на базе Fedora Silverblue/Kinoite), вы можете установить менеджер пакетов Nix поверх этой системы.

Так как Bazzite является иммутабельной ОС с жестким контролем доступа (SELinux), стандартный скрипт установки Nix там не сработает. Специально для таких случаев был разработан **Determinate Nix Installer**. Этот кросс-платформенный установщик официально поддерживает Steam Deck, корректно работает с политиками SELinux и аккуратно разворачивает хранилище `/nix/store`, не конфликтуя с базовым атомарным образом ОС.

Установив Determinate Nix Installer, вы сможете использовать `nix-shell` или `devenv` для создания декларативных сред разработки прямо на Bazzite. Вы сможете точно так же запустить Claude Code внутри Nix-оболочки, чтобы AI помогал вам описывать нужные зависимости в файлах конфигурации, оставляя саму операционную систему нетронутой.

User prompt: Окей проанализируй что будет иметь лучшую совместимость с steamdeck и такой же крутой опыт как на bazzite до всех этих ужасных обновлений

Response: Если искать баланс между максимальной аппаратной совместимостью со Steam Deck OLED и возвращением того «бесшовного» опыта, который радовал вас в ранних версиях Bazzite, стоит обратить внимание на следующие три варианта:

1. Aurora-DX (Самый простой переход и идеальная совместимость UI) Если вы хотите просто переустановить систему и сразу начать работать, Aurora-DX — лучший выбор. Исторически корпорация Valve оптимизирует Steam Deck, его драйверы и интеграцию с игровым режимом эксклюзивно под графическую среду KDE Plasma.

Aurora-DX разрабатывается той же командой Universal Blue, что и Bazzite, но базируется на KDE и позиционируется именно как Developer Experience (стабильная рабочая станция без агрессивных геймерских экспериментов). Вы получите ровно ту же архитектуру атомарных обновлений, встроенный Distrobox, Waydroid и Flatpak, но без графических глитчей GNOME и без навязывания спорных решений, ломающих базовый инструментарий.

2. Jovian-NixOS (Для идеальной декларативной системы под управлением ИИ) Если вы твердо решили отдать управление ОС Claude Code, то проект Jovian-NixOS предоставит самую глубокую интеграцию. Разработчики Jovian специально проделали огромную работу, чтобы портировать модифицированное ядро Valve, прошивки контроллеров и интерфейс Steam UI в парадигму NixOS.

В этом сценарии ИИ сможет управлять вообще всем (от версии ядра до плагинов IDE) через один файл `flake.nix`. Однако у этого пути есть минус: патчи для свежих OLED-реvizий иногда не успевают собираться в публичных бинарных кэшах, из-за чего Steam Deck может периодически тратить часы на самостоятельную компиляцию ядра при обновлениях системы.

3. Базовая SteamOS + Determinate Nix Installer (Гибридный и самый надежный путь) Это компромисс, который дает 100% гарантию совместимости железа и всю мощь Nix. Вы просто оставляете заводскую SteamOS (со всеми гарантиями от Valve на звук, экран и спящий режим).

Сверху вы устанавливаете Determinate Nix Installer — он официально поддерживает Steam Deck и умеет аккуратно разворачивать пакетный менеджер Nix на неизменяемой файловой системе без конфликтов и отключения защиты ядра. При таком подходе ИИ-ассистент будет генерировать для вас конфигурации сред разработки (`devshells`), но системные компоненты (драйверы, Wi-Fi, звук) останутся под надежным управлением Valve.

User prompt: Jovian-NixOS как собрать себе ос скопировав программы для steam deck почему не советуешь chog1 os или как то так игровой дистрибутив с поддержкой gamescope

Response: Вот план исследования для этой темы. Сообщите мне, если что-то надо поменять.

User prompt: Начать исследование

Response: I've completed your research. Feel free to ask me follow-up questions or request changes.

User prompt: Окей давай план для моего агента ии по настройке среды похожей на bazzite но с внедрением jovi nix os и стабильной работе на steam deck на базе nixos конфигурации функционал все должно описано быть для агента ии

Response: Вот архитектурный план и подробные инструкции, которые вы можете скопировать и передать вашему ИИ-агенту (Claude Code) в качестве стартового промпта. План описывает создание декларативной конфигурации NixOS, которая реплицирует консольный опыт Bazzite/SteamOS, но предоставляет гибкую среду разработки.

Системный промпт для ИИ-агента:

«Твоя задача — спроектировать и развернуть конфигурацию NixOS для Steam Deck OLED, используя парадигму Nix Flakes. Система должна загружаться в режиме консоли (Steam UI + Gamescope) аналогично Bazzite, но иметь встроенную среду разработки и поддержку плагинов. Следуй этой архитектурной спецификации:

1. Инициализация Flakes и управление зависимостями

- Создай файл `flake.nix`.
- В блоке `inputs` подключи три основных репозитория: стабильную или нестабильную ветку `nixpkgs`, репозиторий аппаратной поддержки `jovian-nixos` и репозиторий бинарных кэшей `chaotic` (Chaotic-Nyx).
- Обязательно примени директиву `inputs.nixpkgs.follows = "nixpkgs"` для `jovian-nixos`. Это критически важно для предотвращения конфликтов версий системных библиотек и компиляторов в среде разработки.

2. Интеграция бинарных кэшей (Защита от долгих компиляций)

- Внедри модуль `chaotic.nixosModules.default` в конфигурацию системы.
- Настройка бинарного кэша обязательна: без использования готовых кэшей сборка NixOS на Steam Deck попытается локально скомпилировать кастомное ядро Linux от Valve, что займет несколько часов вычислительного времени на мобильном APU устройства.

3. Аппаратная конфигурация (Hardware Enablement)

- Импортируй модуль `jovian-nixos.nixosModules.default` в основной файл `configuration.nix`.
- Включи главный декларативный переключатель: `jovian.devices.steamdeck.enable = true`. Это действие автоматически подтянет все необходимые патчи ядра, драйверы аудио, конфигурации контроллеров и параметры управления питанием, специфичные для Steam Deck OLED.

4. Игровой режим и Steam UI (Gamescope Session)

- Для репликации поведения Bazzite (прямая загрузка в консольный интерфейс) активируй параметры `jovian.steam.enable = true` и `jovian.steam.autoStart = true`.
- примени системные твики планировщика от Valve с помощью флага `jovian.steamos.useSteamOSConfig = true`.
- Назначь пользователя по умолчанию для автоматического входа в игровую сессию.

5. Интеграция Desktop Mode и Decky Loader

- Добавь в конфигурацию графическую среду (рекомендуется KDE Plasma для максимальной совместимости с утилитами Valve).
- Свяжи Steam UI с режимом рабочего стола переменной `jovian.steam.desktopSession = "plasma"` (или `"gnome"`), чтобы переход из игрового режима в режим ПК работал корректно.
- Включи поддержку сторонних плагинов с помощью параметра `jovian.decky-loader.enable = true`, что позволит безопасно развернуть экосистему Decky в защищенной (read-only) файловой системе NixOS.

6. Файловая система и управление состоянием

- Используй утилиту `disko` для декларативной разметки NVMe-накопителя, оставив достаточно места на root-разделе (ext4 или btrfs) для установки тяжелых пакетов среды разработки.

После создания структуры файлов, проверь конфигурацию и предложи команду для сборки, например: `sudo nixos-rebuild switch --flake .#deck.`»

User prompt: Что скажешь по поводу видео <https://youtu.be/ttOs5iWgNzk>

Response: Видео по ссылке `ttOs5iWgNzk` называется "Steamdeck as a Desktop", и его автор — техноблогер Крис Титус (Chris Titus Tech).

В этом видео (и в его сопутствующей статье) наглядно демонстрируется ровно тот гибридный подход, который мы обсуждали: установка пакетного менеджера Nix поверх заводской SteamOS. Главный посыл автора заключается в том, что для создания полноценной рабочей среды на Steam Deck категорически не стоит модифицировать системные файлы, так как любое официальное обновление от Valve просто сотрет все изменения.

Вместо этого в видео показывается изящный обходной путь для развертывания Nix:

1. Пакетный менеджер устанавливается таким образом, чтобы все его данные хранились в домашнем разделе пользователя (создается директория `/home/nix`), который не затрагивается обновлениями системы.
2. Чтобы система корректно видела пакеты, Крис показывает, как создать специальные службы `systemd` (такие как `nix-directory.service` и `nix.mount`). Эти службы при каждой загрузке консоли монтируют пользовательскую папку в корневой каталог `/nix`.

3. После этого вы можете искать нужные пакеты на `search.nixos.org` и устанавливать их через терминал с помощью команды `nix-env`.

Это отличное практическое руководство. Оно подтверждает, что использование Nix позволяет обойти ограничения неизменяемой (read-only) файловой системы без огромных накладных расходов на контейнеры Flatpak. Таким образом, вы получаете мощную среду разработки, а игровой режим SteamOS остается абсолютно нетронутым и безопасным.

User prompt: Как это реализовать и что насчет обновлений steam os которые все сбрасывают

Response: Проблема обновлений SteamOS заключается в том, что система использует механизм A/B разделов: при каждом обновлении корневая файловая система (/) полностью перезаписывается, уничтожая любые пакеты, установленные классическим способом. Однако пользовательский раздел /home и директория с настройками /etc остаются нетронутыми и сохраняются при любых апдейтах.

Реализация "неубиваемого" пакетного менеджера Nix строится именно на использовании этой особенности файловой системы:

1. **Хранение данных:** Все загружаемые пакеты, инструменты разработки и сам Nix физически устанавливаются в защищенную пользовательскую директорию /home/nix.
2. **Службы systemd:** В директории /etc/systemd/system/ (которая не стирается при обновлениях) создаются три специальные системные службы.
3. **Процесс загрузки:** При каждом включении консоли первая служба временно снимает блокировку файловой системы (steamos-readonly disable), создает в корне пустую папку /nix и сразу возвращает защиту от записи. Вторая служба выполняет bind mount (проброс), монтируя вашу сохраненную директорию /home/nix в эту свежесозданную системную папку /nix. Третья служба перезапускает необходимые процессы, чтобы оболочка увидела установленные программы.

В результате NixOS работает так, как ему нужно (обращаясь к пути /nix), но фактически все данные лежат в безопасности в вашем домашнем каталоге.

Вам не обязательно прописывать все эти скрипты вручную. Современный инсталлятор **Determinate Nix Installer** имеет встроенную поддержку архитектуры Steam Deck. При запуске он автоматически настраивает все необходимые службы systemd и пробросы директорий. Благодаря этому ваша декларативная среда разработки гарантированно переживет любое официальное обновление SteamOS от Valve.

User prompt: Nix os и nix store и nix flakes и nix configs это лучше потому что локальный пользователь или хуже я запутался четко разграничить что теряем что получаем где лучше совместимость и надежность работоспособности программы это лучше чем distrobox или похоже на него или как вообще розясни подробности

Response: Понимаю вашу путаницу. Экосистема Nix имеет довольно специфическую терминологию. Давайте разложим все по полочкам: что есть что, как это отличается от Distrobox и почему гибридный подход (SteamOS + локальный Nix) может быть лучшим выбором.

1. Расшифровка терминов (Словарь Nix)

- **Nix (Пакетный менеджер):** Это программа-установщик (как apt в Ubuntu, pacman в Arch или brew), но работает она принципиально иначе.
- **Nix Store (/nix/store):** Это "склад" на вашем диске. В обычных Linux программы разбросаны по всей системе (/usr/bin, /lib и т.д.), из-за чего возникают конфликты версий. Nix складывает каждую установленную программу и ее зависимости в отдельную изолированную папку на складе, присваивая ей уникальный криптографический хэш (например, /nix/store/1234abc...-python-3.11).
- **Nix Configs (Конфиги):** Текстовые файлы (обычно с расширением .nix), в которых вы описываете, какие программы вам нужны. Вы не говорите системе "установи", вы просто перечисляете список в файле, а Nix сам сверяет этот список с тем, что есть на складе.
- **Nix Flakes:** Это современный стандарт конфигов Nix. Их главная фишка — строгая фиксация версий (lock-файлы). Если среда разработки скомпилировалась у вас по Flake-файлу сегодня, она с вероятностью 100% так же скомпилируется через год на другом компьютере.
- **NixOS:** Полноценная операционная система, где *вообще всё* (от драйверов и ядра до графического интерфейса) управляется менеджером Nix через конфигурационные файлы.

2. В чем разница: Nix на SteamOS vs Distrobox на Bazzite?

Distrobox (на Bazzite): Это использование контейнеров (на базе Podman/Docker). Когда вы запускаете Distrobox, вы создаете внутри Steam Deck изолированную "виртуальную коробку" (например, с Ubuntu). Внутри нее вы пользуетесь обычным apt, ставите пакеты.

- *Минусы:* Это дополнительный слой абстракции. Программы работают внутри контейнера, и если контейнер сломается из-за кривого обновления внутри него, его придется удалять. Иногда бывают сложности с пробросом специфического железа (например, USB-дебаггеров) внутрь контейнера.

Локальный Nix (на SteamOS): Вы не используете виртуальные коробки или контейнеры. Установщик Determinate Nix Installer просто добавляет пакетный менеджер Nix прямо в оригинальную SteamOS.

- *Как это работает:* Пакетный менеджер устанавливается так, что его хранилище (/nix/store) физически лежит в вашей домашней папке /home/nix. Обновления SteamOS (от Valve) полностью затирают системные разделы, но они никогда не трогают пользовательский раздел /home. Специальные скрипты при загрузке консоли "привязывают" вашу папку /home/nix обратно к системе.

- **Плюсы:** Программы работают напрямую (bare-metal), у них есть прямой доступ ко всем ресурсам Steam Deck, железу и портам, но их файлы жестко заперты на "складе" Nix и не загрязняют базовую ОС.

3. Что вы получаете и что теряете (Итог)

Если вы возвращаетесь на заводскую SteamOS и устанавливаете туда локальный Nix (Determinate Installer) для создания среды разработки с помощью Claude Code:

Что вы получаете (Плюсы):

1. **Эталонная стабильность оборудования:** Вы возвращаете идеальный игровой опыт. Сон (suspend), вывод изображения, драйверы, HDR, Wi-Fi и звук работают безупречно, потому что ядро и графический стек контролируются напрямую Valve.
2. **Неубиваемая среда разработки:** Все ваши компиляторы, IDE и утилиты хранятся в домашнем разделе и переживут любые мажорные обновления SteamOS (даже переход на новые версии Arch).
3. **Идеально для ИИ:** Claude Code сможет редактировать ваши Flake-файлы, декларативно настраивая окружение, при этом он физически не сможет сломать саму операционную систему.
4. **Никаких контейнеров:** Разработка ведется "на железе", без накладных расходов и багов Distrobox.

Что вы теряете (Минусы):

1. **Порог вхождения:** Устанавливать программы через терминал придется иначе. Вместо привычного `brew install` или `pacman -S` нужно будет добавлять пакеты в `.nix` файл или использовать команду `nix-env / nix profile`.
2. **Системные модификации:** Локальный Nix работает только на уровне пользователя (программы, среды, утилиты). Если вам для разработки нужно будет глубоко модифицировать саму ОС (например, внедрить кастомное ядро или изменить низкоуровневые правила контроллеров `udev`), локального Nix для этого не хватит — для таких глобальных задач потребовалось бы ставить полноценную Jovian-NixOS.

Таким образом, гибридный подход — это золотая середина: Valve заботится о железе и играх, а Nix в домашней папке обеспечивает вас надежным, как швейцарские часы, инструментарием для разработки.

User prompt: Но я теперь незнаю иожет просто поставить nix os с jovix вместо nix

Response: Это совершенно нормальные сомнения — вы стоите перед классическим выбором между абсолютным контролем и прагматичным удобством. Чтобы вам было проще принять окончательное решение, давайте посмотрим на это с практической точки зрения.

Если вы выберете полноценный Jovian-NixOS: Вы полностью удаляете заводскую систему от Valve. Теперь ваша консоль — это чистый лист, который работает исключительно

по правилам «Nix Way», где всё управляется декларативно.

- **Плюсы:** У вас будет идеальная, математически выверенная система. Вы сможете управлять вообще всем (от драйверов до плагинов рабочего стола) из одного конфигурационного файла.
- **Минусы:** Вы становитесь системным администратором своей консоли. Так как архитектура Nix требует сборки пакетов, при неудачном стечении обстоятельств (или отсутствии нужного кэша) консоль может начать компилировать кастомное ядро Linux, что займет часы. Кроме того, при выходе новых игр или прошивок вам придется ждать, пока сообщество Jovian адаптирует патчи Valve под NixOS.

Если вы выберете гибрид (SteamOS + Determinate Nix Installer): Вы оставляете консоль в заводском виде, а среду разработки изолируете в домашнем разделе.

- **Плюсы:** Это самый безболезненный путь. Вы получаете гарантию того, что игровой режим, сон, звук и контроллеры будут работать безупречно (за это отвечает Valve). При этом установщик Nix корректно разместит свое хранилище в директории /home/nix и создаст нужные службы systemd, чтобы все ваши инструменты разработки гарантированно переживали любые обновления корневой файловой системы SteamOS. Вы сможете так же использовать современные функции вроде Nix Flakes и Home Manager для настройки окружения.
- **Минусы:** У вас не будет контроля над системными компонентами ОС, так как базовая система остается неизменяемой.

Что в итоге выбрать?

- Выбирайте **Jovian-NixOS**, только если вы хотите глубоко изучить саму операционную систему NixOS, вас не пугает возня с системными конфигурациями и вы готовы тратить время на обслуживание ОС.
- Выбирайте **SteamOS + локальный Nix**, если ваша главная цель — просто писать код, использовать ИИ-агентов для настройки среды и стабильно играть, не переживая о том, что очередное обновление системы сломает вам звук или Wi-Fi. Для 90% разработчиков гибридный вариант является наиболее надежным и сохраняет кучу времени.

User prompt: Но ведь на nixos можно бекапить как батя и в случае чего легко откатиться разве не кайф и надежно как швейцарские часы так что давай конфиг nixos под steam deck oled

Response: Да, откаты (rollbacks) — это фундаментальное преимущество NixOS. Каждое изменение конфигурации сохраняется в изолированном хранилище /nix/store и генерирует новое поколение (generation) системы. Если обновление сломает графический стек или драйверы, вы просто выбираете предыдущий пункт в меню загрузчика при включении консоли, и система мгновенно откатывается к рабочему состоянию с побитовой точностью.

Вот готовый шаблон декларативной конфигурации, который ваш ИИ-агент (или вы сами) может использовать для создания идеальной среды на базе Jovian-NixOS.

1. Файл `flake.nix`

Этот файл объявляет, откуда скачивать пакеты, и фиксирует их версии.

```
{
  description = "Steam Deck OLED Developer Flake";

  inputs = {
    nixpkgs.url = "github:nixos/nixpkgs/nixos-unstable";

    # Подключение репозитория Jovian
    jovian.url = "github:Jovian-Experiments/Jovian-NixOS";
    jovian.inputs.nixpkgs.follows = "nixpkgs";
  };

  outputs = { self, nixpkgs, jovian, ... }@inputs: {
    nixosConfigurations = {
      deck = nixpkgs.lib.nixosSystem {
        system = "x86_64-linux";
        specialArgs = { inherit inputs; };
        modules = [
          jovian.nixosModules.default
          ./hardware-configuration.nix
          ./configuration.nix
        ];
      };
    };
  };
}
```

Директива `follows = "nixpkgs"` здесь критически важна: она заставляет модули Jovian использовать те же версии системных библиотек и компиляторов, что и ваша основная система, предотвращая дублирование зависимостей и возможные конфликты в среде разработки.

2. Файл `configuration.nix`

Здесь описано само состояние операционной системы.

```
{ config, pkgs, inputs, ... }:

{
  # Включение аппаратных оптимизаций Steam Deck
  jovian.devices.steamdeck.enable = true;
  jovian.steamos.useSteamOSConfig = true;

  # Настройка игрового режима (Steam UI + Gamescope)
  jovian.steam = {
    enable = true;
    autoStart = true;
  };
}
```

```
user = "deck"; # Замените на ваше имя пользователя
desktopSession = "plasma";
};

# Интеграция плагинов
jovian.decky-loader.enable = true;

# Включение режима рабочего стола KDE Plasma
services.xserver.enable = true;
services.desktopManager.plasma6.enable = true;

# Базовая настройка пользователя
users.users.deck = {
  isNormalUser = true;
  extraGroups = [ "wheel" "networkmanager" ];
};

# Сетевые службы и Bluetooth
networking.hostName = "deck";
networking.networkmanager.enable = true;
hardware.bluetooth.enable = true;

# Звуковая подсистема
security.rtkit.enable = true;
services.pipewire = {
  enable = true;
  alsa.enable = true;
  pulse.enable = true;
};

system.stateVersion = "24.05";
}
```

Как это работает:

- Опция `jovian.devices.steamdeck.enable` автоматически скачивает и применяет все необходимые патчи ядра Linux от Valve, решая проблемы с аудиокодеками, контроллерами и управлением питанием на Steam Deck OLED.
- Блок `jovian.steam` запускает микрокомпозитор Gamescope и переводит систему в консольный режим Big Picture прямо при загрузке, копируя поведение заводской ОС. Указание `desktopSession = "plasma"` гарантирует, что кнопка "Перейти на рабочий стол" корректно запустит нативную для Valve среду KDE.
- `jovian.decky-loader.enable` позволяет легально и безопасно развернуть экосистему плагинов Decky на неизменяемой корневой файловой системе.

Важное предупреждение для развертывания: Поскольку NixOS собирает конфигурацию из исходников или бинарных кэшей, первая компиляция кастомного модифицированного ядра Valve непосредственно на процессоре Steam Deck может занять несколько часов. При

реальной настройке вам или вашему агенту будет необходимо добавить в конфигурацию сторонние бинарные кэши (например, `chaotic-nux`), чтобы консоль скачала уже скомпилированное ядро вместо его локальной сборки.

User prompt: Непонял как это компилироваться или взять бинарик готовый в чем разница и почему нельзя для всего готовый бинарик

Response: В NixOS система управления пакетами работает принципиально иначе, чем в привычных дистрибутивах (вроде Ubuntu или обычной SteamOS). Давайте разберем, почему возникает этот нюанс с компиляцией.

Как работают бинарники и компиляция в NixOS:

По своей природе Nix — это система, которая собирает программы из исходного кода. Каждой программе, библиотеке и ядру присваивается уникальный криптографический хэш (идентификатор), который зависит от каждой строчки кода и каждого флага компиляции.

Но чтобы пользователи не ждали сутками, пока их система скомпилируется, существуют **бинарные кэши** (серверы-склады). Для 99% стандартных программ (KDE Plasma, Firefox, системные утилиты, драйверы) официальные серверы NixOS уже всё скомпилировали. Когда вы просите Nix установить эти программы, он сверяет хэш, видит совпадение на официальном сервере и просто **скачивает готовый бинарник** за пару секунд.

Почему нельзя взять готовый бинарник для ядра Steam Deck "из коробки"?

Проблема кроется в том, что ядро Linux со специфическими патчами от Valve (которое предоставляет модуль Jovian-NixOS) — это нестандартный, глубоко модифицированный код.

1. Официальные серверы NixOS не собирают и не хранят готовые бинарники для таких узкоспециализированных пользовательских модификаций.
2. Когда ваш Steam Deck читает конфиг и видит требование "установить ядро Valve", он обращается к официальному серверу. Сервер отвечает: "У меня нет готового файла с таким хэшем".
3. Поняв, что готового бинарника нет, Nix делает единственное, что ему остается: скачивает сырой исходный код ядра и начинает его компилировать локально, прямо на мобильном процессоре вашего Steam Deck. Это невероятно тяжелая математическая задача, которая занимает несколько часов.

Как это решается на практике (Chaotic-Nyx)

Поскольку никто не хочет компилировать ядро часами, на помощь приходит комьюнити. Проект под названием **Chaotic-Nyx** (от создателей репозитория CachyOS и Chaotic-AUR) предоставляет свой собственный сторонний сервер с бинарным кэшем.

У этих ребят есть мощные серверные фермы. Они заранее скачивают исходный код Jovian-NixOS, компилируют тяжелое ядро Valve у себя на мощных процессорах и выкладывают результат.

Если вы добавите их репозиторий в свой файл `flake.nix` (как это было в примере кода), происходит магия: Ваш Steam Deck обращается уже не только к официальному серверу, но и к серверу Chaotic. Он спрашивает: "Есть готовое ядро от Valve?". Сервер Chaotic отвечает "Да, держи", и ваш Steam Deck просто скачивает готовый скомпилированный файл за одну минуту.

Итог: "Готовый бинарник" для всего получить можно, но поскольку инфраструктура NixOS децентрализована, для нестандартных хакерских вещей (вроде переноса системы от консоли на Nix) вам нужно просто указать системе пальцем на тот сервер комьюнити, где добрые люди уже скомпилировали этот код за вас.

User prompt: Окей что на счет окружения `gnome kde` или может вообще `sway` или еще какой модный компожитор на `wayland`

Response: В NixOS прелесть заключается в том, что вы можете менять графическое окружение буквально одной строчкой кода, не засоряя систему. Но для Steam Deck есть критически важные нюансы, связанные именно с форм-фактором устройства.

Давайте разберем все варианты от самых простых к самым хардкорным:

1. KDE Plasma (Выбор по умолчанию) Это "родная" среда, под которую Valve изначально спроектировала весь десктопный опыт SteamOS.

- **Плюсы:** Интеграция работает из коробки. Виртуальная клавиатура корректно вызывается комбинацией кнопок на геймпаде, масштабирование экрана (DPI) настроено под 8-дюймовый дисплей, а тачпад-трекпады работают максимально адекватно.
- **Вердикт:** Если вам нужна бесперебойная среда, которая просто работает как часы — оставайтесь на Plasma.

2. GNOME Jovian-NixOS полностью поддерживает GNOME. Чтобы на него переключиться, достаточно заменить в конфиге `plasma` на `gnome` (`jovian.steam.desktopSession = "gnome"`) и включить соответствующий системный сервис.

- **Плюсы:** GNOME с его крупными жестами и интерфейсом исторически гораздо лучше приспособлен для сенсорных экранов, чем KDE.
- **Минусы:** Настройки интеграции от Valve туда не перенесены. Вам, скорее всего, придется самостоятельно создавать скрипт или искать расширение (которое часто ломается при обновлениях GNOME), чтобы вернуть ярлык "Вернуться в игровой режим" на рабочий стол.

3. Sway и Hyprland (Тайлинговые Wayland-композитеры) Установить модный Hyprland на Jovian-NixOS легко: нужно добавить модуль Hyprland и указать `desktopSession = "hyprland"`. Но здесь вас ждет столкновение с суровой реальностью портативного гейминга:

- **Проблема управления:** Тайлинговые оконные менеджеры созданы для управления исключительно горячими клавишами. На Steam Deck без физической клавиатуры вам будет невероятно сложно переключать фокус между окнами, менять их размер или переходить по рабочим пространствам (воркспейсам) с помощью стиков и тачпада.

- **Виртуальная клавиатура:** В Hyprland или Sway нет встроенной экранной клавиатуры. Вам придется самостоятельно декларировать в Nix установку сторонних программ (например, `wvkbd` или `maliit`) и писать сложные скрипты для их вызова по нажатию кнопок.
- **Автономность и батарея:** Базовые настройки Hyprland с прозрачностью, размытием (`blur`) и тенями быстро разрядят батарею вашего Steam Deck OLED. Вам потребуется глубоко погружаться в конфиги для энергосбережения (например, принудительно отключать эффекты и прописывать `misc:vfr = true`, чтобы экран не рендерил лишние кадры, когда окна неподвижны).

Итог: Если ваш Steam Deck большую часть времени стоит в док-станции, подключенный к внешнему OLED-монитору, механической клавиатуре и мыши — **Hyprland** станет невероятно быстрой, красивой и продуктивной средой для разработки.

Но если вы планируете пользоваться режимом рабочего стола, лежа на диване с консолью в руках (портативный режим), использование тайлинговых композиторов вроде Sway/Hyprland превратится в пытку. Для портативного использования лучше оставить **KDE Plasma**.

User prompt: Отлично, я такие хочу.иметь по одному выбранному инструменту лично мной для каждой задачи отдельно что бы можно было своб клавиатуру виртуальную настроить так как мне хочется .не дублировать програмы с одним и тем же функционалом и жестко прописать конфиги что бы не збивались к определенным программам один рпз настроил и забыл + ты говоришь что hyperland заточен под управление с клавиатуры а steam input просто бесподобный разве он не поможет обойти ограничения управления с слоями и сменными режимами помоему можно настроить даже лучше чем стандартные hotkeys

Response: Вы абсолютно правы насчет Steam Input — это потрясающе мощный инструмент, и ваша идея реализовать управление тайлинговым оконным менеджером (Hyprland) через него не просто жизнеспособна, но и уже успешно применяется энтузиастами. Философия «один инструмент для одной задачи» с жесткой фиксацией конфигурации идеально ложится на парадигму NixOS.

Вот как технически работает связка Steam Input и Hyprland без физической клавиатуры:

1. Навигация через Steam Input (Слои и радиальные меню) Вам не нужны стандартные хоткеи, потому что вы можете настроить Desktop Configuration прямо в клиенте Steam. Сенсорные трекпады Steam Deck позволяют создавать радиальные виртуальные меню (до 16 кнопок на каждый трекпад). Задние лепестки-эспандеры (L4, L5, R4, R5) можно жестко забиндить на модификаторы: Super, Alt, Ctrl, Shift. Таким образом, зажав лепесток Super и сдвинув стик вправо, вы отправляете в систему сигнал Super + Right. Конфигурация Hyprland мгновенно переведет фокус на правое окно. Вы также можете настроить отдельные Action Sets (наборы действий) под разные задачи и переключаться между ними на лету, а запуск программ повесить на легковесный лаунчер вроде rofi, который отлично управляется с крестовины (D-Pad). Вы настраиваете этот маппинг один раз, и он работает на системном уровне.

2. Решение проблемы виртуальной клавиатуры Здесь подход «один надежный инструмент» раскрывается лучше всего. Использовать стандартную клавиатуру SteamOS (по

сочетанию Steam+X) в Hyprland не стоит. Из-за строгих правил безопасности Wayland она часто работает с ошибками: клавиатура отрисовывается поверх окон, но перехватывает фокус на себя, и текст не попадает в адресную строку или терминал.

Идеальным Unix-way решением для вас станет wvkbd — минималистичная виртуальная клавиатура, созданная специально для тайлинговых Wayland-композиторов. Она работает как часы: не перехватывает фокус некорректно и умеет автоматически сдвигать открытые окна вверх (shrinking), чтобы не перекрывать вам поле ввода текста. Ее вызов можно назначить на кастомную команду и повесить на одну из кнопок радиального меню Steam Input.

3. Жесткая фиксация через NixOS (Home Manager) Чтобы ничего не сбивалось после обновлений, вы не правите конфиги вручную в скрытых папках. Вы описываете их прямо в вашем Nix-файле через модуль Home Manager. Например, в конфиге вы можете один раз жестко прописать правила для Hyprland под Steam Deck:

- Ограничить анимации и размытие окон (`decoration:blur:enabled = false`), чтобы не высаживать батарею.
- Включить энергосберегающую функцию `misc:vfr = true` (Variable Frame Rate), чтобы графический процессор не рендерил кадры, когда окна терминала и браузера неподвижны.
- Жестко привязать вызов `wvkbd` и `rofi` к нужным командам.

Единственный нюанс для разработчика Стоит учитывать, что при запуске Hyprland напрямую как сессии из-под Gamescope иногда встречаются конфликты XWayland. Например, открытие некоторых окон в полный экран может вызвать сбой фонового процесса Steam.

В остальном ваша задумка абсолютно логична. Если вы готовы потратить один вечер на настройку слоев в Steam Input и прописывание хоткеев в конфигурации Nix, вы получите невероятно быструю, легковесную и персонализированную среду разработки, которая потребляет минимум ресурсов консоли.

User prompt: Если честно опыт gnome мне больше нравится и я устанавливал множество расширений и шейдеров и все работало отлично но я думал hyperland будет покруче gnome в плане быстрогодействия но незнаю можно ли его настроить под управление свайпами и щипками

Response: GNOME действительно является признанным лидером среди графических оболочек Linux по удобству работы с сенсорными экранами «из коробки», и если вам важен готовый функционал расширений, это прекрасный выбор.

Однако в плане быстрогодействия, плавности и экономии ресурсов Hyprland на голову превосходит GNOME. Hyprland — это минималистичный композитор, который не обременен тяжелыми фоновыми службами полноценной среды рабочего стола, что оставляет максимум вычислительной мощности и заряда батареи для ваших рабочих задач.

Что касается управления пальцами по экрану Steam Deck: **да, Hyprland можно детально настроить под свайпы и щипки именно по сенсорному дисплею.**

Хотя "из коробки" Hyprland отлично распознает жесты с трекпадов (в версии 0.51 разработчики сильно обновили нативную поддержку многопальцевых свайпов и щипков для тачпадов), для работы непосредственно с **сенсорным экраном** (touchscreen) в экосистеме существует мощный плагин под названием **hyprgrass**.

Используя hyprgrass, вы можете один раз жестко прописать в конфигурации NixOS любые реакции системы на ваши касания экрана:

- **Щипки (Pinch):** Например, настроить сведение трех пальцев (pinchin или pinchout) для мгновенного закрытия окна, масштабирования или перевода программы в полноэкранный режим.
- **Свайпы от края (Edge swipes):** Вы можете привязать вызов виртуальной клавиатуры (например, wvkbd) или меню приложений к смахиванию от левого или верхнего края экрана.
- **Многопальцевые свайпы:** Переключение между рабочими пространствами (воркспейсами) свайпом тремя или четырьмя пальцами.
- **Долгие нажатия (Long press):** Функция изменения размера плавающих окон путем долгого удержания пальца на их границе.

В парадигме NixOS интеграция этого плагина означает, что вы описываете свои любимые жесты в конфигурационном файле один раз. После этого ваша настроенная сенсорная среда разработки на базе Hyprland будет запускаться всегда с побитовой точностью, обеспечивая футуристичный опыт управления, который по отзывчивости превосходит даже стандартный GNOME.

User prompt: Говорят есть прикольный shojiWM можем ли мы интегрировать в nixos на steam deck с hyperland

Response: ShojiWM — это действительно очень интересный и визуально впечатляющий проект, но здесь есть один важный архитектурный нюанс. Вы **не сможете** использовать его *вместе* с Hyprland.

В экосистеме Wayland дисплейный сервер и оконный менеджер объединены в одну сущность, которая называется композитором (compositor). И Hyprland, и ShojiWM являются полноценными, независимыми Wayland-композиторами. Вы можете установить их оба в вашу систему NixOS, но запускать их придется по отдельности, выбирая нужную сессию (среду) перед входом в режим рабочего стола.

Что такое ShojiWM и почему он так популярен? Это современный Wayland-композитор, ядро которого написано на Rust (с использованием библиотеки Smithay), но вся логика управления окнами, панелями и поведением настраивается исключительно с помощью скриптов на TypeScript (tsx).

Его главная "киллер-фича" — мощный API для кастомных шейдеров (custom shader pipeline API). Именно благодаря ему в сети завирусились ролики, где рабочий стол выглядит как реалистичное «жидкое стекло» (liquid glass), по которому стекают капли дождя или идет преломление света за окнами терминала. В нем можно легко переключаться между классическим плавающим (floating) режимом окон и тайлинговым (tiling).

Можно ли развернуть его на Steam Deck через NixOS? Да, философия NixOS позволяет легко стянуть исходный код с GitHub (разработчик публикует проект под ником bea4dev) и собрать его под ваше устройство. ShojiWM даже поддерживает специфичные для Linux-гейминга вещи, такие как screen tearing (разрывы кадров для снижения задержки) и работу с xwayland-satellite.

Стоит ли использовать его на Steam Deck (вместо Hyprland)? Если консоль постоянно подключена к розетке и монитору, это прекрасный выбор для создания футуристичного интерфейса. Но для портативного использования есть две критические проблемы:

1. **Батарея и видеопамять:** Рендеринг сложных шейдеров преломления света и прозрачности на рабочем столе требует колоссальных ресурсов. Отрисовка "жидкого стекла" будет постоянно нагружать графическое ядро процессора Steam Deck и может потреблять гигабайты драгоценной VRAM, моментально высаживая батарею консоли.
2. **Сенсорное управление:** В отличие от Hyprland, для которого уже написаны готовые инструменты вроде hyprgrass для щипков и свайпов, в ShojiWM вам придется писать логику обработки сенсорного экрана и интеграцию с виртуальными клавиатурами самостоятельно на TypeScript.

Итог: ShojiWM — это потрясающий проект для визуальной кастомизации. Вы вполне можете добавить его в свою конфигурацию Nix как запасной рабочий стол для работы от док-станции. Но в качестве основного, легкого и энергоэффективного тайлингового менеджера, который будет послушно управляться через Steam Input, Hyprland пока остается намного более зрелым инструментом.

User prompt: Ну, к примеру, мы будем использовать искусственный интеллект для написания на тайпскрипте, как это визуальный эффект. Там, во-вторых, сложные визуальные эффекты мы использовать не будем в том же гноме, я говорю, а я использовал множество шейдеров для открытия, закрытия окна, э анимации окна, не знаю, там прозрачности и размытие тоже. Э и всё это было, и всё это работало, и батарея сильно не садилась, поэтому я хотел бы также И вообще, так как там OLED-экран, я буду придерживаться строгих контрастных чёрных фонов и белых э букв э чтобы максимально использовать эффективность OLED-дисплея и э максимально низкое энергопотребление. К тому же, я полагаю, что у меня останется возможность регулировать TDP, так как это было в SteamOS. Э и э это позволит мне снизить либо отключить некоторые эффекты, которые сильно грузят систему. То есть это не проблема. Нужно экспериментировать и проверять, пробовать. Просто так говорить, что будет высаживаться батарея в ноль - это, ну, при нерациональном использовании ресурсов Steam Deck'a. То есть опять же-таки, можно настроить эти скрипты так, чтобы они не рендерились на неактивных окнах, на неактивных вкладках, на неактивных рабочих столах, а рендерились исключительно на одном экране вот, котором который передо мной. Чтобы не тратить опять же-таки колоссальных ресурсов, которых и так очень мало у Steam Deck'a.

Response: Ваш подход звучит невероятно продуманно, и вы абсолютно правы: если подойти к настройке ресурсов рационально, то ShojiWM может стать отличным полем для экспериментов на Steam Deck.

Вот почему ваша идея технически жизнеспособна и как ее можно реализовать в среде NixOS:

1. Идеальная синергия с OLED-экраном Ваша мысль использовать строгие черные фоны — это прямое попадание в архитектуру OLED. В отличие от IPS-экранов, где подсветка работает всегда, матрица OLED физически отключает пиксели при отображении истинного черного цвета (True Black, #000000). Если ваш ИИ напишет скрипт, где интерфейс и терминал базируются на черном цвете, это сэкономит огромный процент заряда батареи на аппаратном уровне, компенсируя затраты на рендеринг композитора.

2. Умный рендеринг через TypeScript ShojiWM был создан именно для таких задач. Это Wayland-композитор, в котором вся логика управления окнами и серверными декорациями пишется на TypeScript (tsx), а визуальные эффекты реализуются через мощный API кастомных шейдеров. С помощью ИИ вы сможете написать логику, которая будет проверять состояние окна. Например: если окно не в фокусе (неактивно) или перекрыто другим, скрипт просто отключает применение шейдера `liquid glass` для этого конкретного окна. Это кардинально снизит нагрузку на графическое ядро процессора.

3. Управление TDP в Jovian-NixOS Ваше предположение совершенно верно — при переходе на NixOS вы **не теряете** контроль над энергопотреблением. В модулях Jovian-NixOS есть специальный параметр `jovian.devices.steamdeck.enablePerfControlUdevRules`. Когда он включен (а он включен по умолчанию при выборе профиля Steam Deck), система создает правила `udev`, которые открывают доступ к аппаратному управлению лимитом TDP, частотами GPU и яркостью дисплея на уровне пользователя.

Это означает, что у вас будет два пути управления мощностью:

- **Через плагины:** Вы можете использовать экосистему Decky Loader и установить плагин SimpleDeckyTDP, который прекрасно работает в Nix-среде.
- **Через системные службы:** В экосистеме Linux для портативных консолей активно используется утилита `ryzenadj`, которая позволяет напрямую отправлять команды в SMU (System Management Unit) процессора AMD для безопасного андервольтинга и жесткого ограничения TDP (например, залочить консоль на 7-10 Ватт при программировании). Вы сможете написать `bash`-скрипт или службу `systemd` в вашем файле конфигурации `.nix`, которая будет автоматически снижать TDP при запуске сессии ShojiWM.

Экспериментировать с этим — это как раз то, ради чего создавался Linux. Вы можете декларативно описать ShojiWM и все нужные скрипты TypeScript в файле Flake, ограничить TDP до комфортного минимума, использовать отключенные пиксели OLED и наслаждаться футуристичной средой разработки, подогнанной лично под вас.

User prompt: Чем отличается nix os от gix os

Response: GNU Guix (часто называемый Guix System) и NixOS — это два дистрибутива-побратима. Они построены на одной и той же фундаментальной идее: декларативное управление конфигурацией, неизменяемость и хранение изолированных пакетов с криптографическими хэшами (в `/nix/store` и `/gnu/store` соответственно).

Однако между ними есть несколько радикальных архитектурных и философских отличий:

1. Язык конфигурации

- **NixOS** использует собственный функциональный язык выражений (Nix). Это специализированный язык (DSL), который был придуман исключительно для этого пакетного менеджера, и его синтаксис нужно изучать с нуля.
- **Guix** использует Guile Scheme — полноценный диалект классического языка программирования Lisp. Вся система, от сборки пакетов до системных служб, написана на одном мощном языке, что делает код очень элегантным для тех, кто знаком с Lisp.

2. Идеология и проприетарное ПО (Ключевое отличие)

- **NixOS** прагматичен. По умолчанию он ставит свободное ПО, но позволяет легко установить драйверы NVIDIA, закрытые прошивки (firmware), Discord или клиент Steam, просто добавив в конфиг флаг разрешения несвободного ПО (allowUnfree).
- **Guix** — это официальный проект Фонда свободного программного обеспечения (GNU). Он фанатично следует идеологии свободного ПО: в его официальных репозиториях категорически запрещены любые программы с закрытым исходным кодом, а по умолчанию используется ядро linux-libre (очищенное от проприетарных драйверов оборудования). Чтобы установить туда Steam или заставить работать проприетарный Wi-Fi модуль, придется подключать неофициальные сторонние репозитории (например, канал nonguix).

3. Система инициализации (Init System)

- В **NixOS** используется стандартный systemd, к которому привыкло большинство пользователей Linux.
- В **Guix System** от systemd отказались в пользу собственной системы под названием GNU Shepherd, которая также полностью написана и конфигурируется на языке Guile.

4. Размер экосистемы и скорость

- Репозиторий **NixOS** (nixpkgs) огромен и поддерживается огромным сообществом, предлагая более 80 000 пакетов. Если в мире есть какая-то утилита для разработки, она почти гарантированно уже упакована для NixOS.
- У **Guix** сообщество меньше, пакетная база скромнее, а пользователи часто отмечают, что вычисление конфигураций и загрузка пакетов в Guix происходят заметно медленнее, чем в Nix.

Что это значит для вашего Steam Deck OLED: Для ваших задач выбор однозначно должен пасть на **NixOS** (и проект Jovian-NixOS). Портативные консоли вроде Steam Deck критически зависят от закрытых прошивок Wi-Fi/Bluetooth, микрокода AMD, специфических драйверов Valve и, конечно же, самого закрытого клиента Steam. Пытаться развернуть всё это на Guix System означало бы постоянную борьбу с фундаментальной философией дистрибутива, тогда как в NixOS это решается интеграцией нужных модулей без ограничений.

User prompt: Поищи может есть готовые flakes для запуска на steam deck oled

Response: Да, сообщество уже создало множество готовых конфигураций (flakes) для Steam Deck на базе NixOS. Поскольку архитектура NixOS позволяет разработчикам легко делиться всем состоянием системы в виде кода на GitHub, вы можете найти как минимальные шаблоны, так и готовые комплексные проекты.

Вот несколько самых примечательных готовых решений:

1. Официальный фундамент: Jovian-NixOS (Jovian-Experiments/Jovian-NixOS) Это главный репозиторий, который предоставляет базовые модули для работы железа Steam Deck (включая OLED) в NixOS. Вместо того чтобы изобретать велосипед, большинство разработчиков просто импортируют этот репозиторий в свои конфигурации, так как он уже содержит ядро Valve и интеграцию Gamescope.

2. Готовый комбайн: SteamNix (SteamNix/SteamNix) Это отличный пример того, как кто-то уже сделал за вас всю сложную работу. Проект позиционирует себя как "опыт SteamOS на NixOS". Если вы клонируете этот flake, вы сразу получите:

- Вырезанную тяжелую графическую среду — в качестве оконного менеджера используется только легковесный Gamescope.
- Встроенную систему плагинов Decky Loader.
- Игровое ядро CachyOS со встроенным планировщиком BORE для максимальной производительности.
- Автоматически установленные сборки ProtonGE для запуска Windows-игр.

3. Персональные сборки с Hyprland и GNOME Многие энтузиасты публикуют свои «dotfiles» (файлы конфигурации), которые можно взять за основу для ваших экспериментов:

- В репозитории **nomadics9/NixOS-Flake** лежит готовая конфигурация NixOS с уже настроенным Hyprland и панелью Waybar. Это отличная шпаргалка для того, чтобы посмотреть, как люди настраивают Wayland-композиторы на консоли.
- В репозитории **heywoodlh/nixos-configs** есть файл `steam-deck.nix`, где автор реализовал интеграцию среды GNOME поверх базы Jovian-NixOS (переход в GNOME происходит по нажатию кнопки «Switch to Desktop»).

4. Минималистичный гибрид: deck-flake (Stream5710/deck-flake) Если вы всё же решите остаться на стандартной заводской SteamOS, но захотите использовать Nix для сред разработки, этот репозиторий содержит базовые конфиги Nix и Home Manager, адаптированные для безопасного развертывания поверх операционной системы Valve.

Вы можете клонировать любой из этих репозиторийев (с помощью `git clone`) и использовать их `flake.nix` как отправную точку, выбросив лишнее и добавив свои собственные скрипты.

User prompt: Как мне поступить сейчас я хочу готовый nixos настроен и заточен под steam deck на hyperland настроенный тоже утилитами для настройки жестов

работающим steam input. Кастомным ядром и тд и тп . сейчас стоим bazzite распиши пошаговую инструкцию не придумывая все заново а испльзуюя github сообщество шаблонов конфигураций

Response: Переход с Bazzite на полностью настроенный NixOS с поддержкой Jovian, Hyprland и сенсорных жестов требует использования готовых конфигураций (Flakes). Сообщество уже проделало большую часть работы по интеграции этих инструментов. Вот пошаговое руководство, основанное на шаблонах GitHub-проектов, таких как Jovian-NixOS, SteamNix и hyprgrass.

Шаг 1. Подготовка установочного носителя

1. Скачайте минимальный установочный образ (Minimal ISO) с официального сайта NixOS и запишите его на USB-накопитель.
2. Подключите USB-накопитель к Steam Deck. Для первой установки настоятельно рекомендуется подключить внешний монитор и клавиатуру, так как загрузчик может не вывести изображение на встроенный экран корректно.
3. Выключите Steam Deck, зажмите кнопку «Громкость Вниз» и нажмите кнопку питания, чтобы выбрать загрузку с USB.

Шаг 2. Инициализация структуры Flake Загрузившись в систему, подключитесь к Wi-Fi (используя команду `iwctl` или `nmtui`). Активируйте поддержку Flakes и создайте директорию для ваших конфигураций. Вы можете скомбинировать базовый репозиторий железа Jovian-NixOS и плагин `hyprgrass` для сенсорного экрана.

Создайте файл `flake.nix`, используя наработки сообщества:

```
{
  description = "Steam Deck OLED - Jovian + Hyprland + Hyprgrass";

  inputs = {
    nixpkgs.url = "github:nixos/nixpkgs/nixos-unstable";
    jovian = {
      url = "github:Jovian-Experiments/Jovian-NixOS";
      inputs.nixpkgs.follows = "nixpkgs";
    };
    hyprland.url = "github:hyprwm/Hyprland";
    hyprgrass = {
      url = "github:horriblename/hyprgrass";
      inputs.hyprland.follows = "hyprland";
    };
  };

  outputs = { self, nixpkgs, jovian, hyprland, hyprgrass, ... }@inputs: {
    nixosConfigurations.deck = nixpkgs.lib.nixosSystem {
      system = "x86_64-linux";
      specialArgs = { inherit inputs; };
      modules = [
        jovian.nixosModules.default
```

```

        ./configuration.nix
        ./hardware-configuration.nix
    ];
};
};
}

```

Шаг 3. Настройка системы (configuration.nix) Создайте файл `configuration.nix`, опираясь на практики интеграции Jovian. В нем активируется профиль Steam Deck, кастомное ядро от Valve, Gamescope и указывается Hyprland как среда рабочего стола:

```

{ config, pkgs, inputs, ... }:

{
    # Активация оптимизаций и ядра Steam Deck
    jovian.devices.steamdeck.enable = true;
    jovian.steamos.useSteamOSConfig = true;

    # Настройка игрового режима
    jovian.steam = {
        enable = true;
        autoStart = true;
        user = "deck";
        desktopSession = "hyprland";
    };

    # Включение Hyprland
    programs.hyprland = {
        enable = true;
        package = inputs.hyprland.packages.${pkgs.system}.hyprland;
    };

    # Установка виртуальной клавиатуры и плагина жестов
    environment.systemPackages = [
        pkgs.wvkbd
        inputs.hyprgrass.packages.${pkgs.system}.default
    ];

    # Базовые настройки (замените на свои при необходимости)
    users.users.deck = {
        isNormalUser = true;
        extraGroups = [ "wheel" "networkmanager" ];
    };
    networking.hostName = "deck";
    networking.networkmanager.enable = true;

    system.stateVersion = "24.05";
}

```

Параметр `jovian.steam.desktopSession = "hyprland"` гарантирует, что кнопка переключения на рабочий стол в Steam UI запустит именно ваш Wayland-комpositor.

Шаг 4. Настройка жестов (Hyprgrass) и Steam Input Конфигурация самого оконного менеджера (в `~/.config/hypr/hyprland.conf`) должна содержать активацию плагина жестов. Проект `hyprgrass` отлично обрабатывает свайпы и щипки на сенсорных экранах мобильных устройств. В конфиг Hyprland добавляются параметры для работы с плагином:

```
plugin = /path/to/hyprgrass.so
```

`plugin:touch_gestures { workspace_swipe_fingers = 3 workspace_swipe_edge = l }` Виртуальную клавиатуру `wvkbd` можно назначить на жест смахивания от края экрана или жестко привязать вызов команды к одной из кнопок радиального меню Steam Input в настройках контроллера.

Шаг 5. Разметка диска и установка Отформатируйте разделы накопителя. Для автоматизации разметки диска сообщество часто применяет декларативную утилиту `disko`, но можно разметить диски и вручную. Сгенерируйте аппаратную конфигурацию (если монтировали разделы в `/mnt`):

```
sudo nixos-generate-config --root /mnt
```

Запустите финальную сборку и установку системы:

```
sudo nixos-install --flake .#deck
```

Поскольку это установка с нестандартным ядром Jovian-NixOS, и если вы не подключили сторонние бинарные кэши (например, Chaotic), установка может занять несколько часов, так как процессор консоли будет самостоятельно компилировать ядро Linux. После успешного завершения и перезагрузки, Steam Deck включится в привычном режиме консоли, из которого вы сможете перейти в настроенную среду Hyprland.

User prompt: Кркто можно мне git ссылку на это добро

Response: Конечно, вот прямые ссылки на репозитории GitHub, которые вам понадобятся для сборки этой системы. Вы можете использовать их как основу для вашего конфигурационного файла:

1. Фундамент для Steam Deck (Ядро и драйверы)

- **Jovian-NixOS:** <https://github.com/Jovian-Experiments/Jovian-NixOS>

Это самый главный репозиторий. Он содержит все патчи от Valve, настройки графического сервера и ядро, необходимые для корректной работы Steam Deck OLED.

2. Готовый шаблон-комбайн (Для клонирования)

- **SteamNix:** <https://github.com/SteamNix/SteamNix>

Отличный готовый проект, который позиционируется как опыт SteamOS на базе NixOS. Он уже настроен на использование Jovian, имеет встроенную систему плагинов Decky

Loader и включает автоматическую загрузку ProtonGE. Вы можете взять его `flake.nix` за основу и просто добавить туда Hyprland.

3. Плагин для сенсорного экрана (Жесты)

- **Hyprgrass:** <https://github.com/horriblename/hyprgrass>

Тот самый плагин для Hyprland, который добавит нативную поддержку многопальцевых свайпов и щипков прямо на экране консоли.

4. Бинарный кэш (Чтобы не компилировать ядро часами)

- **Chaotic-Nyx:** <https://github.com/chaotic-cx/nyx>

Репозиторий, который предоставляет свежие и нестабильные пакеты (например, `mesa_git` или `linux_cachyos`). Добавление его серверов в вашу конфигурацию позволит скачивать готовые бинарные файлы вместо того, чтобы Steam Deck компилировал тяжелое ядро самостоятельно.

Вы можете передать эти ссылки вашему ИИ-агенту вместе с инструкцией из предыдущего шага, и он поможет грамотно объединить их в единую рабочую конфигурацию.
